Ex 3: Generative Adversarial Networks (GANs) for Synthetic Image Generation

Learning Objective:
Implementation of Generative Adversarial Networks (GANs) for Synthetic Image Generation using the MNIST Dataset.

Steps:

1. Load MNIST dataset, normalize images to range -1,1. Split the Batch and shuffle the dataset.
2. Build the Generator
   a. Input: random noise vector (100-dim).
   b. Apply Dense → reshape to $7\times7\times256$.
   c. Apply BatchNorm + LeakyReLU.
   d. Upsample using Conv2DTranspose layers to $28\times28\times1$.
   e. Output activation: tanh.
3. Build the Discriminator
   Input: $28\times28\times1$ image.
   a.Apply Conv2D + LeakyReLU + Dropout layers.
   b.Flatten output.
   c.Final Dense layer with Sigmoid → real or fake probability.
4. Define Loss Functions
   Discriminator:
   i.Real images → label 1
   ii.Fake images → label 0
   Generator:
   i.Fake images should be classified as real (label 1)
   Use Binary Cross-Entropy loss.
5. Define Adam optimizer for both networks.
6. Training Step
   Generate random noise.
   a.Produce fake images using Generator.
   b.Evaluate real and fake images using Discriminator.
   c.Compute Generator and Discriminator losses.
   d.Compute gradients and update weights.
7. Training Loop
   i.Repeat training step for each batch in each epoch.
   ii.Periodically generate and display sample images.
8. Visualize Output by comparing generated fake images with the original images.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Dense, Reshape, Flatten, LeakyReLU
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.optimizers import Adam
data = pd.read_csv("sign_mnist_test.csv")

X = data.iloc[:,1:].values
X = (X.astype('float32') - 127.5) / 127.5
X = X.reshape(X.shape[0], 28, 28, 1)
def build_generator():

    model = Sequential()

    model.add(Dense(256, input_dim=100))
    model.add(LeakyReLU(alpha=0.2))

    model.add(Dense(512))
    model.add(LeakyReLU(alpha=0.2))

    model.add(Dense(1024))
    model.add(LeakyReLU(alpha=0.2))

    model.add(Dense(784, activation='tanh'))
    model.add(Reshape((28,28,1)))

    return model

def build_discriminator():

    model = Sequential()

    model.add(Flatten(input_shape=(28,28,1)))

    model.add(Dense(512))
    model.add(LeakyReLU(alpha=0.2))

    model.add(Dense(256))
    model.add(LeakyReLU(alpha=0.2))

    model.add(Dense(1, activation='sigmoid'))

    return model
```

```
generator = build_generator()
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy',
                      optimizer=Adam(0.0002,0.5),
                      metrics=['accuracy'])

discriminator.trainable = False

gan = Sequential([generator, discriminator])

gan.compile(loss='binary_crossentropy',
            optimizer=Adam(0.0002,0.5))


def train(epochs=10000, batch_size=64):

    half_batch = batch_size // 2

    for epoch in range(epochs):



        idx = np.random.randint(0, X.shape[0], half_batch)
        real_imgs = X[idx]

        noise = np.random.normal(0,1,(half_batch,100))
        fake_imgs = generator.predict(noise, verbose=0)

        d_loss_real = discriminator.train_on_batch(real_imgs,
np.ones((half_batch,1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs,
np.zeros((half_batch,1)))


        noise = np.random.normal(0,1,(batch_size,100))
        valid_y = np.ones((batch_size,1))

        g_loss = gan.train_on_batch(noise, valid_y)
```

```
        if epoch % 1000 == 0:
            print(f"Epoch {epoch}  D Loss: {d_loss_real[0]}  G Loss:
{g_loss}")

            save_images(epoch)


def save_images(epoch):

    noise = np.random.normal(0,1,(25,100))
    gen_imgs = generator.predict(noise)

    gen_imgs = 0.5 * gen_imgs + 0.5

    fig, axs = plt.subplots(5,5)
    count = 0

    for i in range(5):
        for j in range(5):
            axs[i,j].imshow(gen_imgs[count,:,:,0], cmap='gray')
            axs[i,j].axis('off')
            count += 1

    plt.show()


train(epochs=10000, batch_size=64)
```

Output:

Result:

Thus the program for implementation of generative adversarial networks (GANs) for Synthetic Image Generation using the MNIST Dataset has been executed successfully.